

PROJECT I: MEDICAL DECISION SUPPORT APPLICATION PREDICTING HEART FAILURE RISK WITH EXPLAINABLE ML (SHAP)

You have been tasked with developing an advanced clinical decision-support tool to help physicians accurately predict the risk of heart failure in patients. Your solution must be accurate, interpretable, efficient, and professionally deployed.

Project Goals:

- **Develop** a robust, explainable machine learning model.
- **Ensure transparency** of model predictions using SHAP explainability.
- **Create** an intuitive user interface (Streamlit or Flask).
- **Follow professional software development practices** (GitHub, automated CI/CD).
- **Demonstrate** prompt engineering by clearly documenting AI-generated prompts used in your workflow.

Dataset

You will use the following dataset:

<https://archive.ics.uci.edu/dataset/519/heart%2Bfailure%2Bclinical%2Brecords>

Preliminary Data Analysis (Critical Questions):

Clearly document your answers to these questions in your notebook (notebooks/eda.ipynb):

- **Missing values:** Are there any? How will you handle them?
- **Outliers:** Are there outliers, and how will you manage them?
- **Class imbalance:** Is the dataset balanced?
 - **Class distribution:** Approximately 68% survived, 32% deceased (imbalanced).
 - **How to handle imbalance:**
 - Apply oversampling techniques (e.g., SMOTE), undersampling techniques, or class-weighting methods.
 - Document the chosen method clearly in your notebook and justify your choice.
- **Correlation:** Are features highly correlated? Explain your strategy for handling correlated features.

Recommended ML Models (Select at least 3):

Choose and evaluate at least three of the following models:

- **Random Forest Classifier**
- **XGBoost Classifier**
- **LightGBM Classifier**
- **Logistic Regression**

Evaluate performance using ROC-AUC, accuracy, precision, recall, and F1-score.

Clearly document your reasoning for the final selected model.

Additional Required Analyses:

Optimizing Memory Usage:

Some dataset features consume more memory than necessary.

- **Task:** Develop a function (`optimize_memory(df)`) in `data_processing.py` that systematically optimizes memory usage by adjusting data types (e.g., `float64` to `float32`, `int64` to `int32`).
- Clearly demonstrate the memory improvement before and after optimization in your notebook.

SHAP Explainability:

Clearly integrate SHAP visualizations to interpret your model:

- Generate SHAP summary plots.
- Demonstrate feature-level interpretations clearly on your interface.

Interface Development (Streamlit or Flask):

Build a clear, intuitive user interface that:

- Allows physicians to input patient data.
- Displays prediction results clearly.
- Provides easy access to SHAP explanations.

GitHub Repository & Workflow:

Each team should create its own GitHub repository (**no forks**). Structure your repository professionally and include a GitHub Actions workflow (.github/workflows) to automate testing and continuous integration.

Required Project Architecture

Your repository must follow the following structure:

```
project/
├── data/
├── notebooks/
│   └── eda.ipynb
├── src/
│   ├── data_processing.py
│   ├── train_model.py
│   └── evaluate_model.py
├── app/
│   └── app.py
├── tests/
├── requirements.txt
├── Dockerfile
└── README.md
```

Description

- notebooks/ : exploratory data analysis
- src/ : core ML pipeline (data processing, training, evaluation)
- app/ : web interface (Streamlit or Flask)
- tests/ : automated tests
- README.md : documentation of your project

Automated Testing Requirement

Your project must include at least one automated test.

Example structure:

```
tests/
└── test_data_processing.py
```

Example tests:

- verify missing values handling
- verify optimize_memory(df) function
- verify model loading and prediction

These tests should be executed automatically through **GitHub Actions**.

Task Distribution and Management (Using Trello):

To efficiently manage your project, we recommend using Trello:

<https://trello.com/>

- Set up a Trello board for your team.
- Clearly define tasks and responsibilities (e.g., Data preprocessing, Model Training, SHAP analysis, Interface development, etc.).
- Regularly update the Trello board to track progress clearly:
 - **To Do**
 - **In Progress**
 - **Review**
 - **Done**

Prompt Engineering Documentation:

To gain experience in prompt engineering:

- Select **one specific task** from your project (e.g., memory optimization function, data preprocessing, model evaluation).
- Clearly document the exact prompts you used (with Copilot or ChatGPT), and the corresponding results.
- Discuss the effectiveness of your prompts and potential improvements.

Include these details clearly in your project documentation (README.md or dedicated markdown file).

Critical Questions for your README:

Explicitly answer the following questions:

- Was the dataset balanced? If not, how did you handle imbalance? and what was the impact?

- Which ML model performed best? Provide performance metrics.
- Which medical features most influenced predictions (SHAP results)?
- What insights did prompt engineering provide for your selected task?

Final Deliverables Checklist:

Ensure your repository contains:

- Professionally structured code.
- Clearly documented exploratory analysis, class imbalance handling and model training steps.
- Working SHAP explainability integration.
- Intuitive web interface.
- Functional CI/CD pipeline (GitHub Actions).
- Memory optimization function clearly implemented.
- Comprehensive documentation, including your prompt engineering task.
- **Reproducibility**

Your project must be fully reproducible. A user should be able to run the project using:

```
pip install -r requirements.txt
python src/train_model.py
streamlit run app/app.py
```

Your README must clearly explain how to:

- install dependencies
- train the model
- run the application

You have **until Sunday the 15th at noon** to successfully deliver your project.

Good luck!